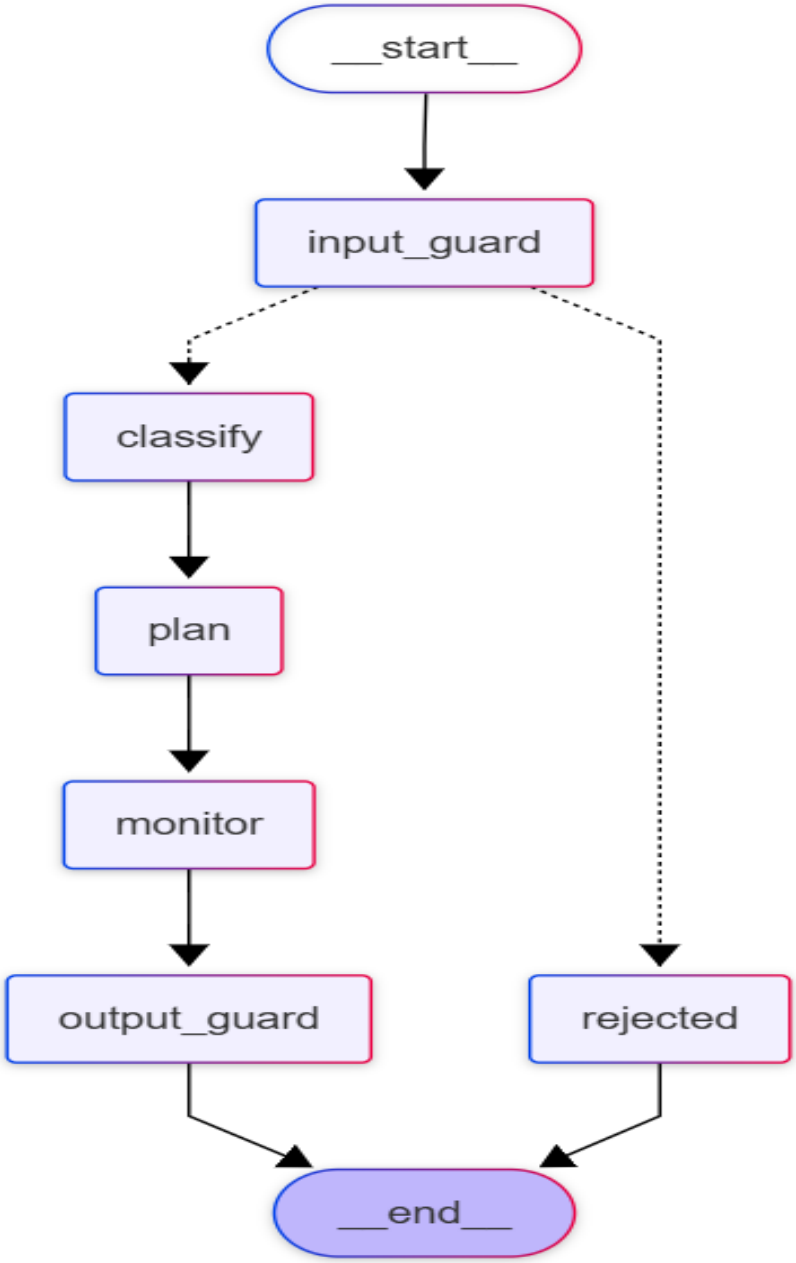
UrbanPulse

LangGraph & LangSmith Pipeline

Detailed Technical Report

This report provides an in-depth explanation of the architectural components, execution order, and inter-component relationships of the LangGraph-based AI workflow in the UrbanPulse project.



Proposed Changes — New Directory Structure

```
ai-service/
├── .env / .env.example
├── Dockerfile
├── pyrightconfig.json
├── requirements.txt
├── run.py                          # Simplified entry point
│
└── src/
    └── urbanpulse/                 # ★ SINGLE package root
        ├── __init__.py
        ├── __main__.py             # [NEW] python -m urbanpulse
        │
        ├── api/                    # ★ FastAPI layer (was: app/)
        │   ├── __init__.py
        │   ├── app.py              # create_app() factory
        │   ├── dependencies.py     # [NEW] verify_internal_secret
        │   └── routes/
        │       ├── __init__.py
        │       ├── health.py       # [NEW] Health endpoint (extracted)
        │       ├── crewai_route.py # CrewAI pipeline endpoint
        │       └── langgraph_route.py
        │
        ├── core/                   # ★ Cross-cutting concerns
        │   ├── __init__.py
        │   ├── config.py           # Pydantic Settings
        │   ├── logging.py          # Structlog setup
        │   └── langsmith.py        # LangSmith init
        │
        ├── models/                 # ★ Split schemas
        │   ├── __init__.py         # Re-exports everything
        │   ├── enums.py            # IncidentCategory, Status, AgentName, AgentAction
        │   ├── incident.py         # IncidentDTO
        │   ├── pipeline.py         # PipelineResult, AgentLogCreate
        │   └── callback.py         # AgentResultCallback, ProcessIncidentRequest, HealthResponse
        │
        ├── services/               # ★ [NEW] Shared business logic
        │   ├── __init__.py
        │   ├── callback.py         # Spring Boot HTTP callback
        │   └── validator.py        # Content consistency checker
        │
        ├── guardrails/             # ★ [NEW] Extracted guardrails
        │   ├── __init__.py
        │   ├── input_guard.py      # Prompt injection detection
        │   └── output_guard.py     # Output safety check
        │
        ├── tools/                  # ★ Shared core tool logic (KEPT)
        │   ├── __init__.py
        │   ├── geocoding.py        ✓ unchanged
        │   ├── infrastructure.py   ✓ unchanged
        │   ├── patterns.py         ✓ unchanged (import path update only)
        │   ├── risk_profile.py     ✓ unchanged
        │   ├── time_context.py     ✓ unchanged
        │   └── weather.py          ✓ unchanged
        │
        ├── crewai_pipeline/        # ★ CrewAI — follows official structure
        │   ├── __init__.py
        │   ├── config/
        │   │   ├── agents.yaml     ✓ unchanged
        │   │   └── tasks.yaml      ✓ unchanged
        │   ├── agents.py           # [NEW] Agent factory methods
        │   ├── tasks.py            # [NEW] Task factory methods
        │   ├── tools.py            # BaseTool wrappers (was: tools/crewai_tools.py)
        │   ├── crew.py             # UrbanPulseCrew class ONLY
        │   └── runner.py           # [NEW] run_pipeline() extracted
        │
        └── langgraph_pipeline/     # ★ LangGraph — follows official structure
            ├── __init__.py
            ├── state.py            # [NEW] PipelineState TypedDict
            ├── nodes/              # [NEW] One node per file
            │   ├── __init__.py
            │   ├── classifier.py
            │   ├── planner.py
            │   ├── monitor.py
            │   └── rejected.py
            ├── tools.py            # @tool wrappers (was: tools/langgraph_tools.py)
            ├── graph.py            # Graph construction ONLY
            └── runner.py           # run_langgraph_pipeline()
```

1. What is LangGraph and Why Was It Used?

LangGraph is a library used to design AI workflows with cyclic behavior and state management. It was chosen for the UrbanPulse project to ensure that incident notifications are processed through a defined logic framework in a safe and controllable sequence of steps.

System Overview

Component	File	Role	Description
Runner	runner.py	Entry Point	Receives raw data and initializes the system
State	state.py	Shared Memory	The data container carried throughout the entire process
Graph	graph.py	Workflow Map	Defines which step runs and when
Nodes & Tools	nodes/ + tools.py	Worker Agents	Functions executing each step and their toolset

2. Component Details

A Runner (runner.py) — System Trigger

The Runner receives IncidentDTO-typed data from the outside world (Frontend/API) and converts it into a State object that LangGraph can process.

- **Memory Initialization:** Prepares the blank slate (Initial State).
- **Async Execution:** Uses asyncio.to_thread to prevent AI operations from blocking the main server thread.
- **Result Transformation:** Reads the populated state from the Graph and converts it into logs to be persisted to the database.

B State (state.py) — Shared Memory

The State acts like a shared form that all agents (nodes) read from and write to. Each agent reads the form, fills in its own section, and passes it on to the next.

- **Data Types:** Contains fields such as incident details, security approvals, category, priority, assigned department, and SLA duration.
- **Consistency:** Ensures data is not lost throughout the process and that every step can access the output of the previous one.

C Graph (graph.py) — Workflow Map

The Graph is the "brain" of the system. It defines the paths (edges) connecting nodes and the decision mechanisms that control routing.

- **Entry Point:** Defines the flow starting at input_guard.
- **Conditional Edges:** If input_guard deems the content unsafe, it routes to the rejected node; otherwise it proceeds to classify.
- **Sequential Flow:** Strictly enforces the sequence: classify -> plan -> monitor -> output_guard.

D Nodes (nodes/ directory) — Worker Agents

Each node is a Python function with a single, well-defined area of expertise.

- **Classifier:** Determines the type of the incident.
- **Planner:** Assigns the responsible department and resolution timeframe.
- **Monitor:** Prepares a professional summary of all actions taken.
- **Guardrails:** Ensures input and output safety.

E Tools (tools.py) — Toolset

External data sources used by agents when making decisions.

- **weather_context_tool:** Fetches the current weather conditions at the incident location.
- **district_risk_tool:** Analyzes the risk profile of the district.
- **infrastructure_tool:** Locates nearby critical infrastructure such as hospitals and fire stations.
- **geolocation_tool:** Converts geographic coordinates into a human-readable address.

3. Execution Order (Step-by-Step)

1	**START**	runner.py is triggered -> Raw data is converted into PipelineState
2	**SECURITY**	input_guard runs -> Content is inspected for safety
3	**REJECTED (Option A)**	If content is risky -> routes to rejected node -> END
4	**ANALYSIS (Option B)**	classify node runs -> Incident is categorized using tools.py
5	**PLANNING**	plan node runs -> Department and SLA are determined

6	**SUMMARIZING**	monitor node runs -> A professional summary of all actions is generated
7	**FINAL CHECK**	output_guard runs -> The generated response is verified to be safe
8	**FINISH**	runner.py receives the result -> Saves to database and returns to the user

4. Co-existence with CrewAI

The project preserves the existing CrewAI infrastructure while adding LangGraph as an alternative, more structured data processing pipeline.

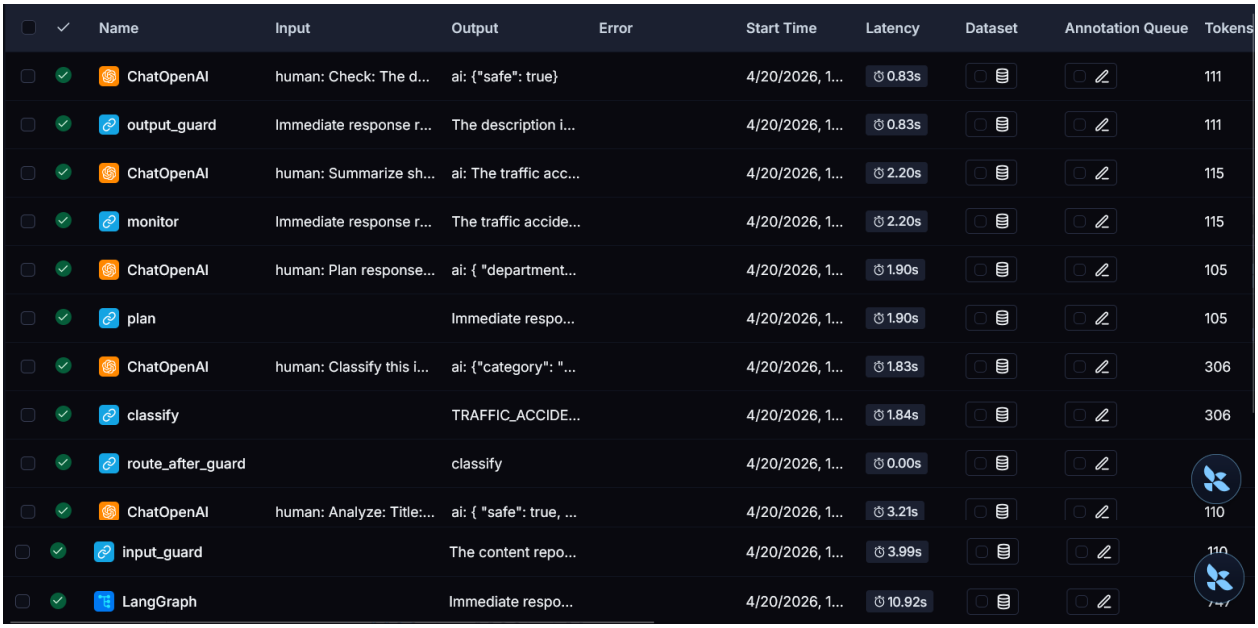
Criteria	CrewAI	LangGraph
Structure	Autonomous agent discussion	Deterministic steps
Use Case	Research and exploration processes	Low-error-tolerance hierarchical flows
Control	Flexible	Strict and hierarchical
Example	Agents deciding among themselves	Security first, then classification

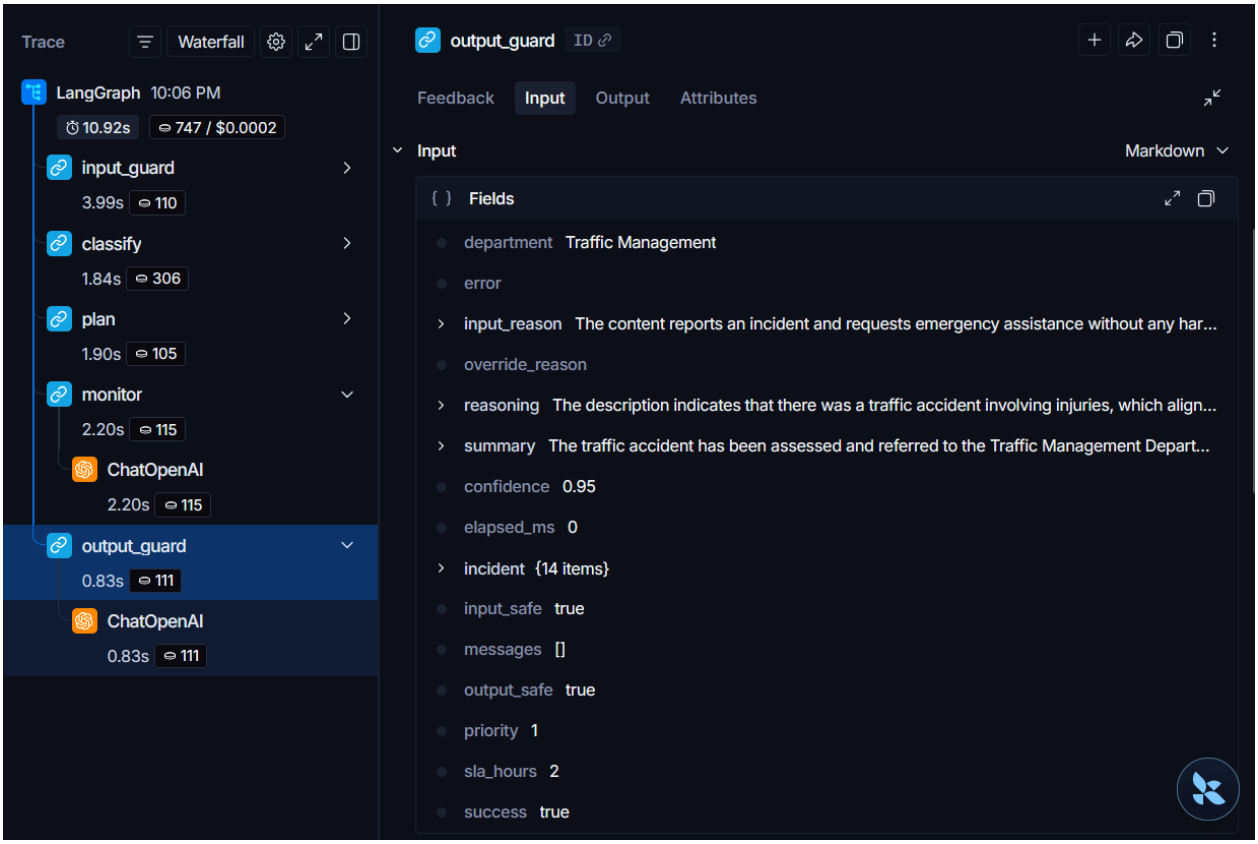
Both systems are integrated within ai-service and can be run independently or in a hybrid fashion.

5. Monitoring with LangSmith

While the system is running, the duration of each step and the number of tokens consumed are automatically sent to the LangSmith dashboard.

- When the Runner is triggered, LangSmith generates a Trace ID for tracking.
- Whenever an agent in the Nodes directory executes, LangChain automatically reports the data to the cloud.
- This means if an error occurs at any step, the exact agent and timestamp can be identified visually.





6. Implemented Features (Summary)

Feature	Description
Input Guardrails	Protection against prompt injection attacks
Smart Logic	Tool-assisted classification using external data sources

Determinism	Steps execute according to the defined map, not by chance
Full Observability	Every millisecond recorded and traceable via LangSmith